

CS1CA Computer Systems Architecture

Data manipulation (1)

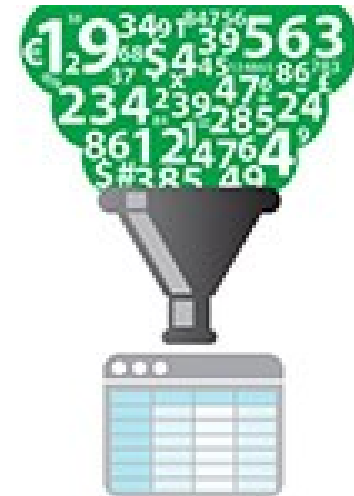
Week 3 lecture 1 of autumn semester 2025

Lectured by

Dr. Hong Wei

h.wei@reading.ac.uk

Room 133, Polly Vacher



The topic will be delivered
in 3 lectures

Contents in the topic

- Computer architecture: introduction
- Memory organisation
- Instruction encoding and decoding
 - Introduction to a simple machine
 - A simple machine language
- Instruction execution cycle

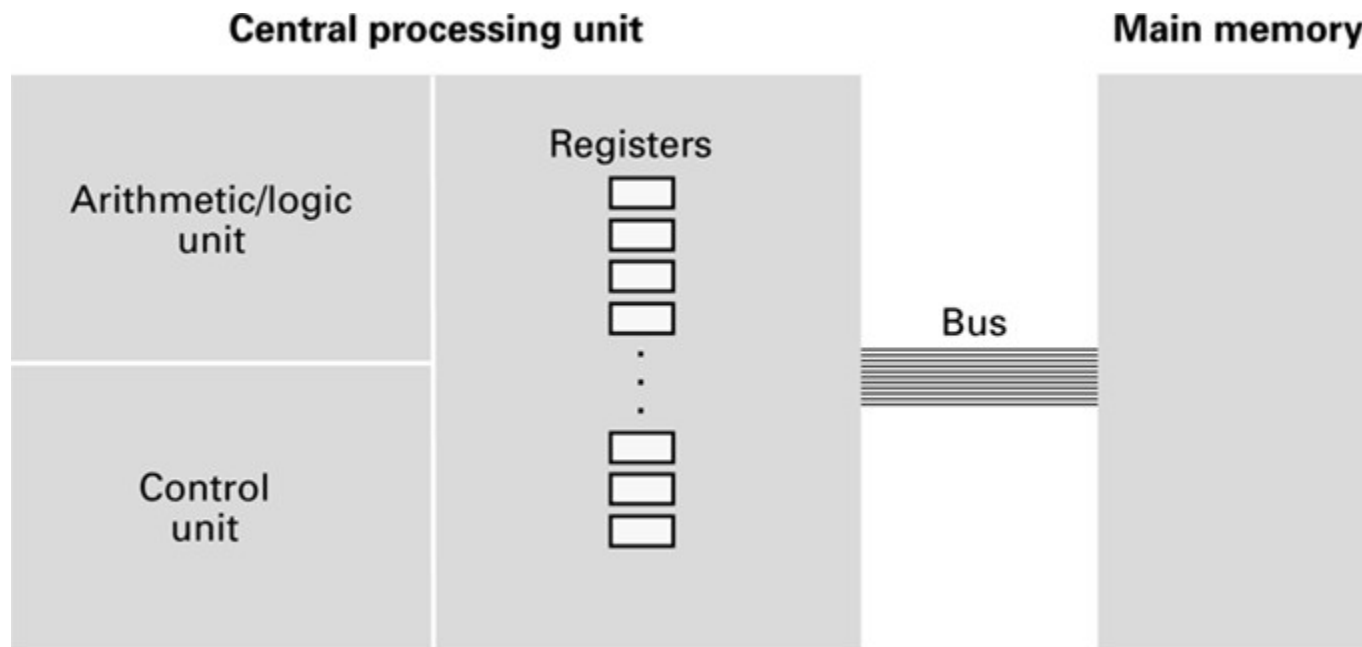
In this lecture

- The following contents are delivered.
 - Computer architecture: introduction
 - Memory organisation
- The learning outcomes are listed below.
 - To be able to describe the functions of CPU and memory in a computer system, and to be aware of different architectures of computer systems
 - To be able to articulate memory organisation, to address memory cells, and to place data in memory cells

Computer architecture: introduction

- A digital electronic computer usually consists of an interconnected system of processors, memories, and input/output devices.
- In this topic, we revisit Central Processing Unit (CPU) and Primary Memory (main memory), and discuss their basic functions.
- It is necessary to understand them for **data manipulation**, which refers to moving data from one location/unit to another as well as performing operations such as arithmetic and logic calculations

CPU, main memory, and bus



- Inside a CPU, there are
 - arithmetic/logic unit (ALU) and control unit (functions?)
 - a set of registers: **general** vs **special purpose registers** (usage?)
- A bus connects the CPU and the main memory for transferring bit patterns between them.

Units and registers in the CPU

- The arithmetic/logic unit (ALU) performs operations on data, such as addition and division.
- The control unit coordinates the machine's activities and directs operations by providing timing and control signals.
- Registers: temporary storage inside the CPU
 - General-purpose registers (GPR): designed as temporary holding places for data being manipulated by the CPU.
 - Special-purpose registers (SPR): designed to keep special information during the operations, such as Program Counter (PC) to keep the memory address where the next instruction to be fetched.

Stored program concept: memory

- The main memory is a temporary storage unit where running programs and relevant data are stored.
- A program can be encoded as bit patterns, called “instructions”.
- The CPU, controlled by signals from the control unit, can fetch instructions from the memory by using **memory address** and execute them one by one inside the CPU.
- The *von Neumann architecture* is built upon the concept of the stored-program process.

Homework: explore more about the *von Neumann architecture*. Any advantages and disadvantages? How about Harvard architecture?

Architecture: CISC vs RISC

- Two fundamental types of CPU Instruction Set architectures: **CISC** and **RISC**
- **CISC**: **C**omplex **I**nstruction **S**et **C**omputer
 - Many instructions; variable length encoding; convenient, but high power-consumption.
 - Example: Intel
- **RISC**: **R**educed **I**nstruction **S**et **C**omputer
 - A few instructions; fixed length encoding; simple and fast; power-efficient.
 - Examples: PowerPC from Apple/IBM/Motorola, and ARM used in most smartphones.

Let's first look at memory organisation

Main memory



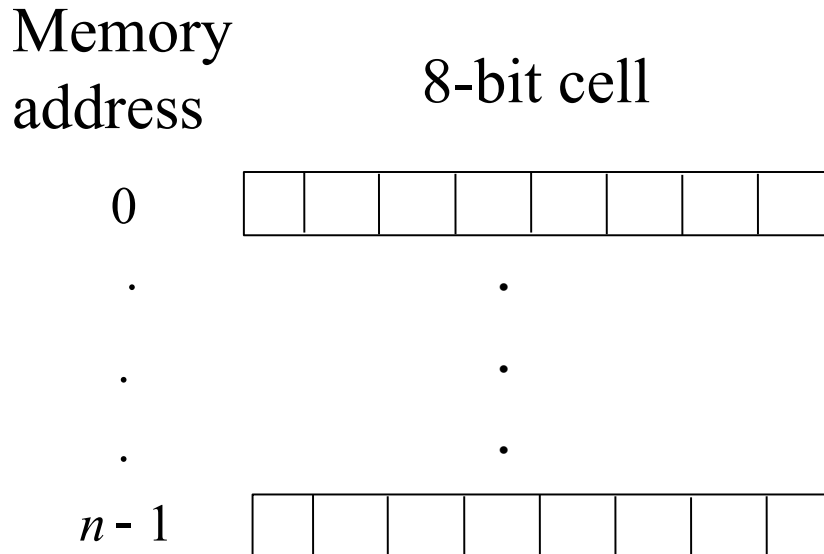
- The main memory is organized in manageable units called **cells**.
- A computer contains a large collection of such cells.
- **Terminology**
 - Random Access Memory (RAM): memory in which individual **cells** can be easily accessed in any order
 - Static RAM (SRAM) and Dynamic RAM (DRAM): different technology and usage

Memory organisation – 1

- Memory consists of a collection of cells, each of which can store a piece of information (instruction or data).
- Each cell has a number allocated, called **memory address**, that the CPU uses in accessing instructions and data stored in the memory during its execution.
- Representing **memory address**: binary/hexadecimal representation from 0 to $n-1$ if a memory has n cells. A cell is the unit of main memory.
- **Size of a cell**: a storage space of k bits, where it can hold any one of 2^k different bit patterns.

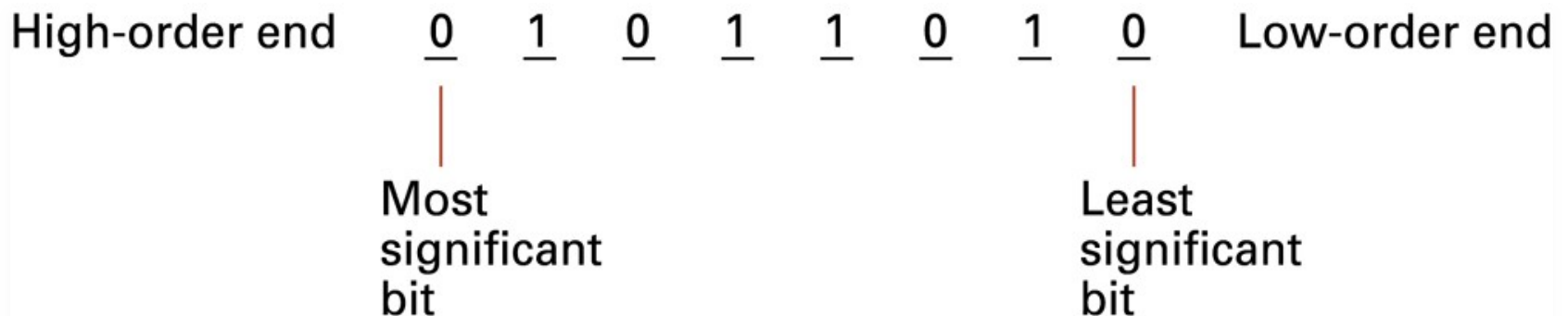
Memory organisation – 2

- Although k could be any integer, almost all computer manufacturers have standardized on an 8-bit cell, *i.e.* $k = 8$.



Each cell can store 8-bit or 1 byte data or instruction.

Organization of a byte-size memory cell



Memory address and address space

- **Memory address:** A “name” that uniquely identifies one cell in the computer’s main memory
 - A name is a number.
 - These numbers are assigned consecutively starting at zero to the maximum (related to the memory address space).
- **Memory address space:** the number of bits used to represent memory addresses determines the overall number of cells that can be addressed. This is referred as the memory address space.
 - If m bits are used to represent memory addresses, there are $n = 2^m$ cells which can be addressed. The address space is from 0 to $n-1$.

Memory space/capacity measures

How many bits should be used for a memory address?

This usually depends on the architecture. A 32-bit architecture uses 32-bits to represent memory address. Hence the address space is from 0 to $(2^{32}-1)$, which can accommodate about 4GB information.

- **Kilobyte (KB):** 2^{10} bytes = 1024 bytes
 - Example: 10 KB $\Rightarrow 10 \times 1024$ bytes
- **Megabyte (MB):** 2^{20} bytes = 1,048,576 bytes
 - Example: 3 MB $\Rightarrow 3 \times 1,048,576$ bytes
- **Gigabyte (GB):** 2^{30} bytes = 1,073,741,824 bytes
 - Example: 4 GB $\Rightarrow 4 \times 1,073,741,824$ bytes

Memory cells arranged by addresses

Assign addresses to

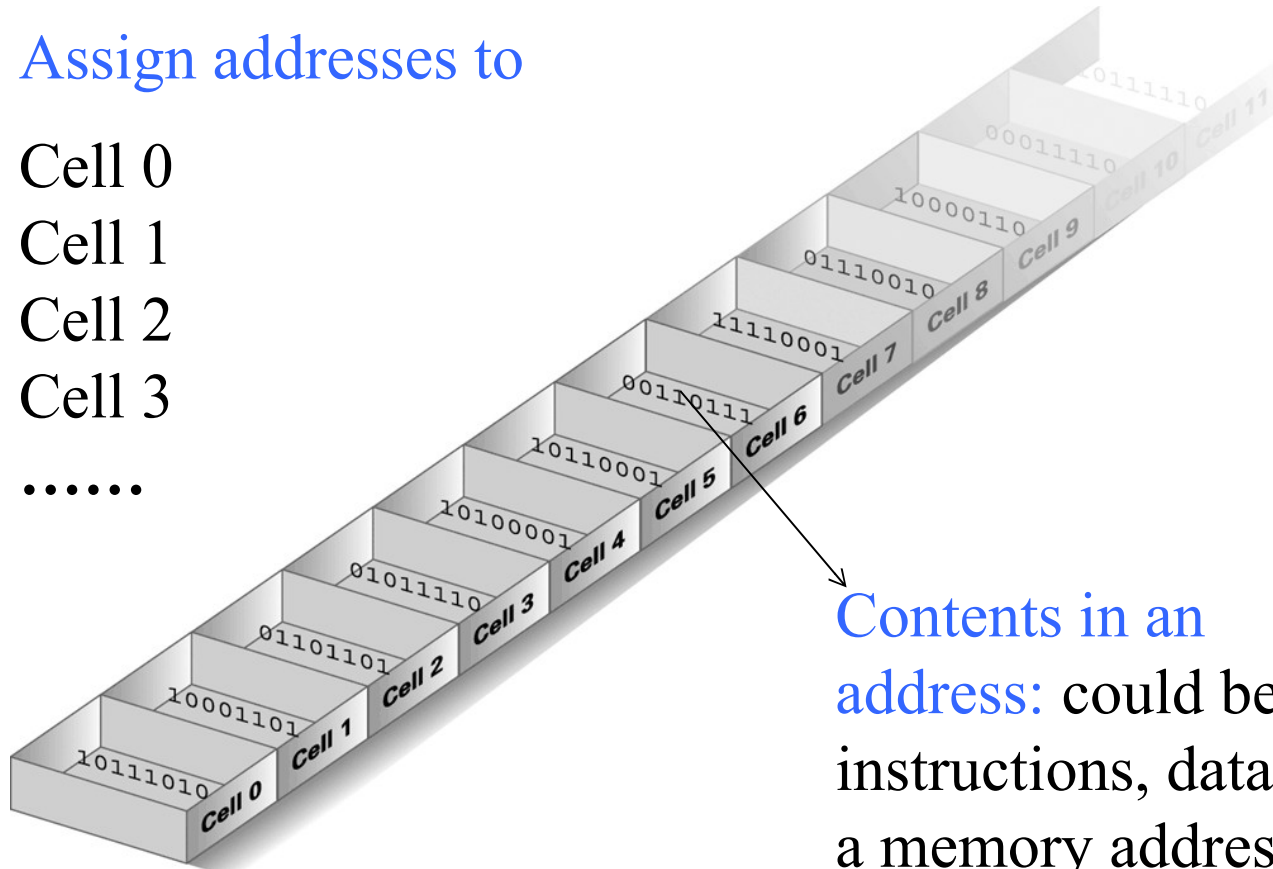
Cell 0

Cell 1

Cell 2

Cell 3

.....



Contents in an address: could be instructions, data or a memory address

Ordering bytes in memory

- As mentioned before, 8 bits construct a byte.
- Bytes are grouped into **words** stored in memory as well as registers.
 - 32 bit architecture has 4 bytes as a word
 - 64 bit architecture has 8 bytes as a word
 - Words are meaningful to CPU (data or instructions)
- **Big endian**: address to the most significant byte when loading/storing a word from/to memory.
- **Little endian**: address to the least significant byte when loading/storing a word from/to memory.

Ordering bytes in memory: example (1)

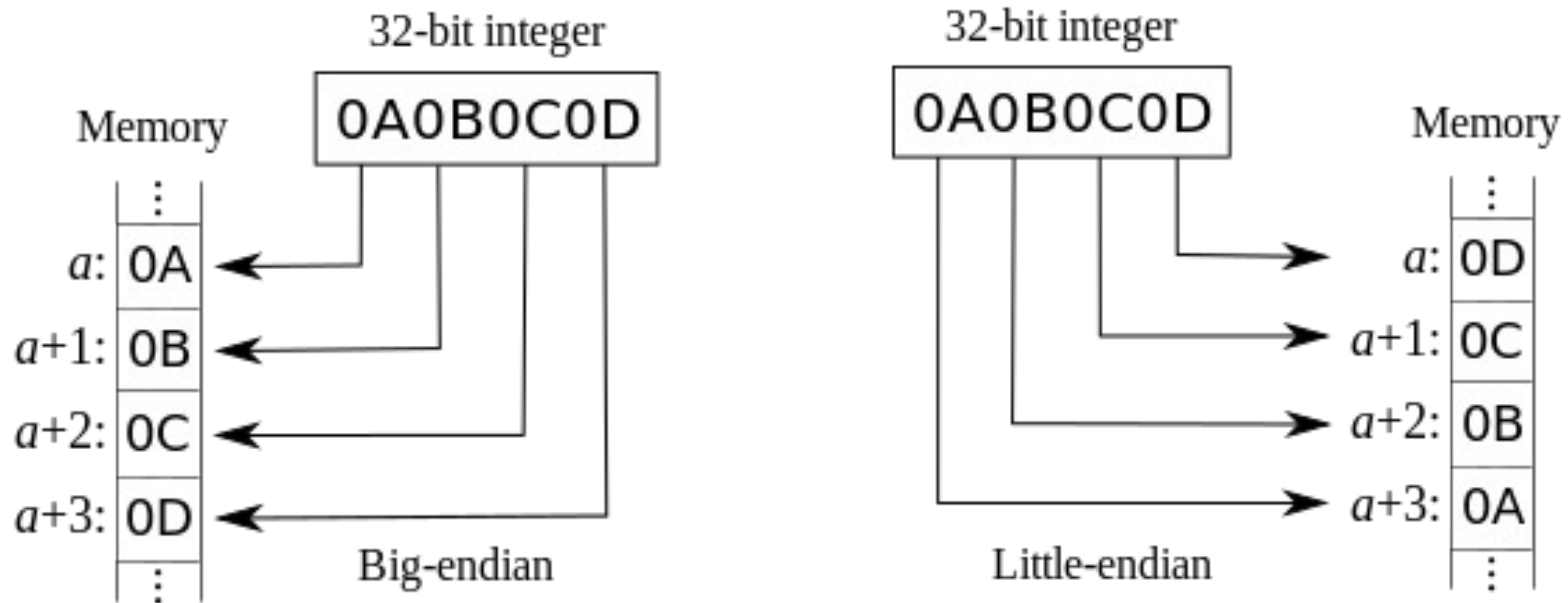
- Load a 32-bit word **34E84652** (4 bytes) into a register from memory cells starting with the address of 0x00, where 0x is a prefix to denote hexadecimal values.
 - “34” is the most significant byte
 - “52” is the least significant byte

Note the byte order in memory cells

Big endian	Memory address	Little endian																
<table><tr><td>3</td><td>4</td></tr><tr><td>E</td><td>8</td></tr><tr><td>4</td><td>6</td></tr><tr><td>5</td><td>2</td></tr></table>	3	4	E	8	4	6	5	2	0x00	<table><tr><td>5</td><td>2</td></tr><tr><td>4</td><td>6</td></tr><tr><td>E</td><td>8</td></tr><tr><td>3</td><td>4</td></tr></table>	5	2	4	6	E	8	3	4
3	4																	
E	8																	
4	6																	
5	2																	
5	2																	
4	6																	
E	8																	
3	4																	
	0x01																	
	0x02																	
	0x03																	

Ordering bytes in memory: example (2)

Store a 32-bit word in a register to memory addresses when CPU addressing at a:



(Courtesy of the internet)

End of the lecture

Questions and exercises

- What are memory address and memory address space?
- A colour image is formatted as a 2D array containing 256×256 pixels in a digital camera. Each pixel has three channels to encode red, green and blue, and 8 bits are used to encode intensity per pixel per channel. The camera has a memory capacity of 2GB. How many images can be stored in the camera?
- A 32-bit word, 22334455 (in hex), is in a register to be stored in memory cells. The CPU addresses the memory at address 0x00 (in hex).
 - What bit pattern is in memory address 0x01 if it is in big endian?
 - What bit pattern is in memory address 0x02 if it is in little endian?